

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Mohd. Razia Alangir Banu
Student Name	Y. Yashwanth
Roll Number	2520030506
Branch	CSE
Course Outcome	CO6 (Applications & Synthesis of Non-Linear Data Structures)

Case Study Topic: Architecting a Real-Time Enterprise Fraud Detection Engine Using Graph Cycle Analysis

Work Done Summary:

A global digital payments network processes millions of peer-to-peer (P2P) banking transactions concurrently. Organized financial syndicates exploit these high-speed platforms by conducting Mule Account Fraud via Circular Routing Loops. In these advanced laundering schemes, funds are broken down and rapidly funneled through a chain of seemingly unrelated placeholder accounts before returning to the original source wallet (e.g., *Account A* $\rightarrow$ *Account B* $\rightarrow$ *Account C* $\rightarrow$ *Account A*). This obscures the audit trail and allows illicit funds to be withdrawn legally.

The risk management division represents accounts as vertices and transactions as directed edges within an massive, fast-evolving transactional graph. The security tier operates under three strict production parameters:

1. **Real-Time Intervention:** The fraud engine must flag and freeze transaction loops within milliseconds of execution to prevent funds from clearing.
2. **Backtracking and Trail Recovery:** Identifying the path isn't enough; the system must accurately record and output the precise sequential path of the laundering cycle for legal compliance reporting.
3. **Scale and Density Handling:** The algorithm must navigate through billions of normal, legitimate transactions without misidentifying standard cross-account transfers as fraudulent cycles.

The engineering team determines that standard linear data structures fail to map multi-account links. They propose a non-linear graph system running an optimized Depth-First Search (DFS) Cycle Detection Subroutine enhanced with a Tri-Color Marking Architecture (White, Gray, Black) to identify and extract illicit financial rings in real-time.

Implementation Details:

1. Inefficiency of Linear Frameworks vs. Graph Topologies

Relational schemas and linear audit files track operations sequentially based on timestamps. Finding a circular loop across multiple accounts (e.g., five hops away) requires executing complex, expensive recursive self-joins across millions of database rows. This degrades read-performance and makes real-time blocking impossible.

By modeling the infrastructure as a non-linear directed graph $G = (V, E)$, structural multi-party dependencies are natively stored in memory. Accounts exist as entities (vertices), and transactions map directly as directed pointers (edges). Tracking money laundering becomes an algorithmic problem of identifying cycles within a directed graph, which can be executed instantly as transactions occur.

2. The Tri-Color Marking Mechanism

Standard DFS loop checks in undirected graphs look for any previously visited node. In dense *directed* graphs, however, hitting a visited node often just means two separate paths converge at the same terminal account, which is completely legitimate. To prevent false positives, we use three distinct operational states:

- **WHITE (Unvisited):** The account has not yet been analyzed by the search pipeline.
- **GRAY (Visiting/In Stack):** The account is currently being evaluated. Its downstream transaction branch is actively being traversed. It is a part of the current active transaction path.
- **BLACK (Fully Processed):** The account and all of its downstream paths have been completely checked with no cycles discovered. It is safe.

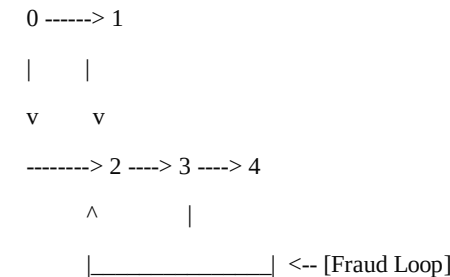
The Fraud Trigger: If the DFS traversal follows a transaction edge and hits a node currently marked as GRAY, it means the path has looped back into an account that is still open and waiting down the current line. This confirms a back-edge exists, revealing a closed circular loop.

3. Step-by-Step Execution Trace

- Given transactions: 0 $\rightarrow$ 1, 0 $\rightarrow$ 2, 1 $\rightarrow$ 2, 2 $\rightarrow$ 3, 3 $\rightarrow$ 4, 4 $\rightarrow$ 2.
- All nodes initialized to WHITE.

Plaintext

Visual Graph Layout:



- Step 1: Start at Node 0. Mark 0 as GRAY.
- Step 2: Move to neighbor 1. Mark 1 as GRAY.
- Step 3: Move to neighbor 2 from 1. Mark 2 as GRAY.
- Step 4: Move to neighbor 3 from 2. Mark 3 as GRAY.
- Step 5: Move to neighbor 4 from 3. Mark 4 as GRAY.
- Step 6: From Node 4, evaluate its outward edge leading back to Node 2.
- Detection: Node 2 is currently marked as GRAY! This indicates a back-edge collision. A laundering ring is detected: 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 2.

Output:-

Java
import java.util.ArrayList;

```

import java.util.Collections;
import java.util.List;

public class FraudDetectionEngine {

    // Definition of the states for Tri-Color Marking
    enum NodeColor {
        WHITE, // Unvisited
        GRAY, // Active in current path / Under investigation
        BLACK // Completely processed with no cycles
    }

    private final int totalAccounts;
    private final List<List<Integer>> adjacencyList;
    private final List<Integer> activePathTrack;
    private boolean cycleDetected = false;

    public FraudDetectionEngine(int accountsCount) {
        this.totalAccounts = accountsCount;
        this.adjacencyList = new ArrayList<>();
        for (int i = 0; i < accountsCount; i++) {
            this.adjacencyList.add(new ArrayList<>());
        }
        this.activePathTrack = new ArrayList<>();
    }

    public void addTransaction(int fromAccount, int toAccount) {
        this.adjacencyList.get(fromAccount).add(toAccount);
    }

    /**
     * Top-level engine call to scan the entire transaction network for cycles.
     */
    public void detectFraudulentLoops() {
        NodeColor[] colors = new NodeColor[totalAccounts];
        for (int i = 0; i < totalAccounts; i++) {
            colors[i] = NodeColor.WHITE;
        }

        System.out.println("Beginning Real-Time Network Graph Analysis...");
        System.out.println("-----");

        for (int account = 0; account < totalAccounts; account++) {
            if (colors[account] == NodeColor.WHITE) {
                if (dfsColorTrace(account, colors)) {
                    cycleDetected = true;
                }
            }
        }

        if (!cycleDetected) {
            System.out.println("SUCCESS: Transaction network is verified clean. No loops found.");
        }
    }
}

```

```

/**
 * Recursive Tri-Color DFS algorithm.
 */
private boolean dfsColorTrace(int current, NodeColor[] colors) {
    // Push node onto the active tracking path
    colors[current] = NodeColor.GRAY;
    activePathTrack.add(current);

    for (int neighbor : adjacencyList.get(current)) {
        // Case 1: Hit a GRAY node - Fraudulent Cycle Confirmed
        if (colors[neighbor] == NodeColor.GRAY) {
            printFraudRingReport(neighbor, current);
            return true;
        }

        // Case 2: Hit an unvisited WHITE node - Traverse deeper
        if (colors[neighbor] == NodeColor.WHITE) {
            if (dfsColorTrace(neighbor, colors)) {
                return true;
            }
        }
        // Case 3: BLACK nodes are skipped immediately as they are verified safe
    }

    // Backtrack: Pop node from path stack and finalize its state to BLACK
    activePathTrack.remove(activePathTrack.size() - 1);
    colors[current] = NodeColor.BLACK;
    return false;
}

/**
 * Extracts and prints the transaction loop path.
 */
private void printFraudRingReport(int loopStartNode, int loopEndNode) {
    System.out.println("!!! SECURITY ALERT: FRAUDULENT CIRCULAR REVENUE RING DETECTED !!!");
    List<Integer> fraudCyclePath = new ArrayList<>();

    // Backtrack through active path trace to extract the loop sequence
    for (int i = activePathTrack.size() - 1; i >= 0; i--) {
        int account = activePathTrack.get(i);
        fraudCyclePath.add(account);
        if (account == loopStartNode) {
            break;
        }
    }
    Collections.reverse(fraudCyclePath);
    fraudCyclePath.add(loopStartNode); // Complete the loop visualization

    System.out.print("Identified Laundering Sequence Flow: ");
    for (int i = 0; i < fraudCyclePath.size(); i++) {
        System.out.print("Account [" + fraudCyclePath.get(i) + "]);");
        if (i < fraudCyclePath.size() - 1) System.out.print(" -> ");
    }
}

```

```

        System.out.println("\nAction Taken: Associated nodes flagged; outbound asset clearance frozen.");
    }

    public static void main(String[] args) {
        // Instantiate graph matching parameters in Question A
        int totalAccounts = 5; // Accounts 0, 1, 2, 3, 4
        FraudDetectionEngine engine = new FraudDetectionEngine(totalAccounts);

        // Populate system with active ledger routes
        engine.addTransaction(0, 1);
        engine.addTransaction(0, 2);
        engine.addTransaction(1, 2);
        engine.addTransaction(2, 3);
        engine.addTransaction(3, 4);
        engine.addTransaction(4, 2); // Creates loop back to Account 2

        engine.detectFraudulentLoops();
    }
}

```

FS

GitHub Link: <https://github.com/Yashwanthwhy/DSA-2/blob/main/Casestudy-6.pdf>

Student Signature	Faculty Signature